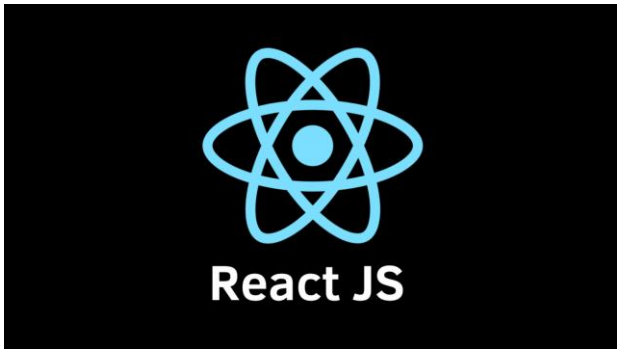
Express JS



Satya Kaveti

NODE.JS **3**

ARCHITECTURE MODEL 4
NODE.JS CONSOLE - REPL 5
NODE.JS BASICS 6
NODE.JS MODULES 7
NPM - NODE PACKAGE MANAGER 9
NODEJS EXAMPLES – MODULE BY MODULE 11
NODE.JS – FRAMEWORKS 15
NODE.JS – DIFFERENCES 16
REF. 16

EXPRESS.JS **17**

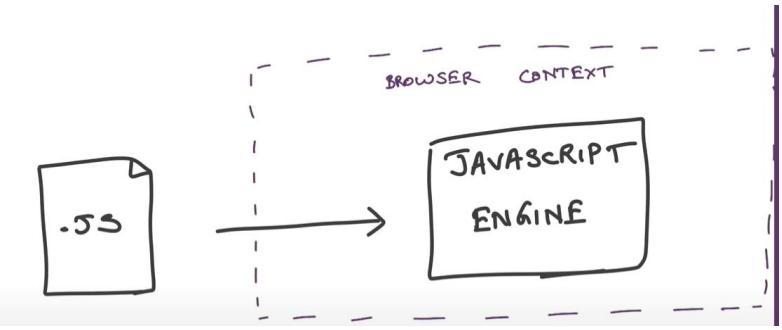
EXPRESS.JS WEB APPLICATION 18
EXPRESS.JS – DATABASE ACCESS 22
NODE.JS - TEMPLATE ENGINES 25

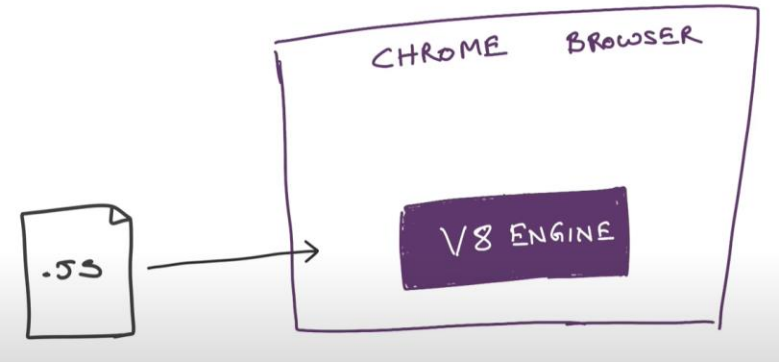
REACT JS **26**

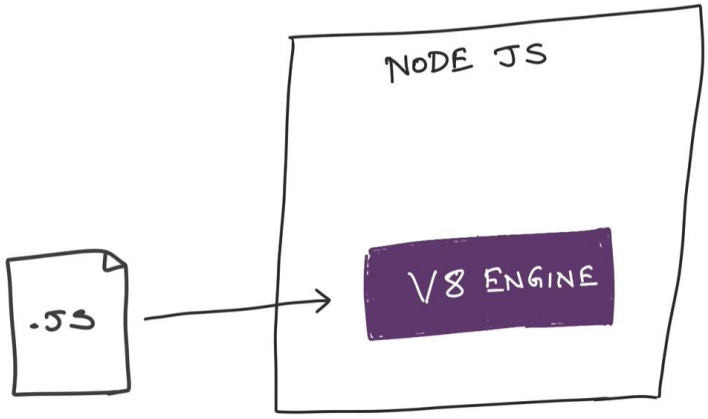
REACT - INTRODUCTION 26
REACT - CREATE REACT APP 28
REACT - JSX - JAVASCRIPT + XML 31
REACT – COMPONENTS 32
REACT – DATA HANDLING 36
LIFECYCLE METHODS 37
REF. 37

Node.js

Node.js is an open-source server-side runtime environment built on Chrome's V8 JavaScript engine. It provides an event driven, non-blocking (asynchronous) I/O and cross-platform runtime environment for building highly scalable server-side applications using JavaScript.





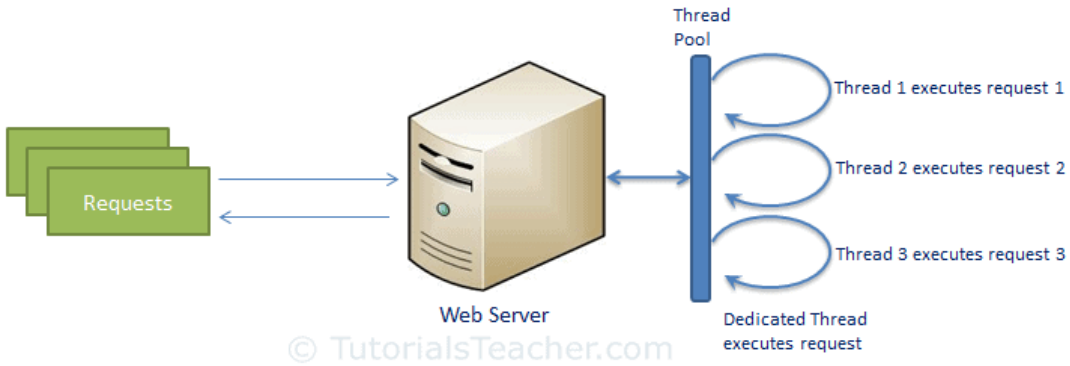


Node.js can be used to build different types of applications such as command line application, web application, real-time chat application, REST API server etc.

Architecture model

Normal Sever Request Processing

In Normal web server model, each request is handled by a dedicated thread from the thread pool. If no thread is available in the thread pool, then the request waits till the next available thread.

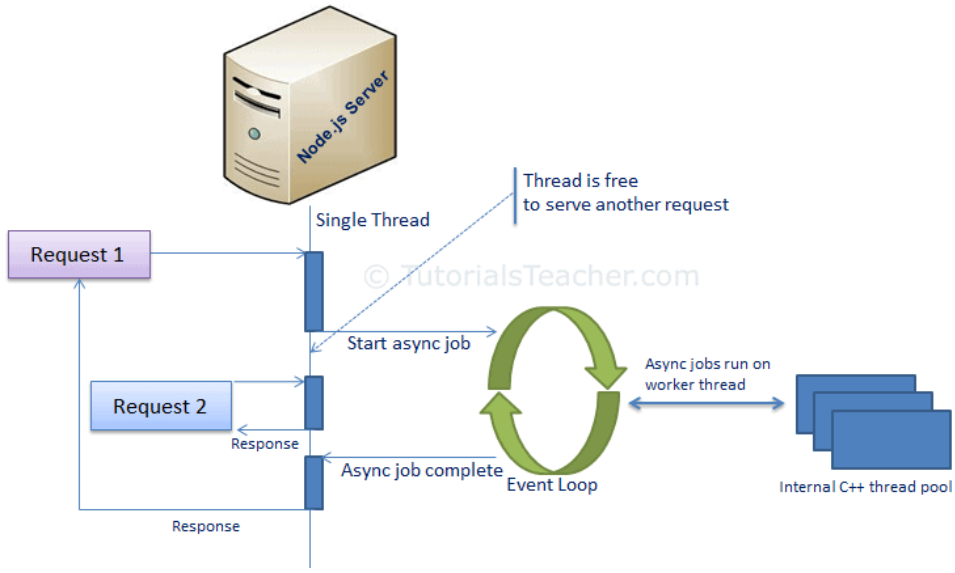


Nodejs Request Processing

Node.js runs in a single process and the application code runs in a single thread. All the user requests to your web application will be handled by a single thread and all the I/O work or long running job is performed asynchronously for a particular request.

So, this single thread doesn't have to wait for the request to complete and is free to handle the next request. When asynchronous I/O work completes then it processes the request further and sends the response.

An event loop is constantly watching for the events to be raised for an asynchronous job and executing callback function when the job completes.



Node.js is not fit for an application which performs CPU-intensive operations like image processing or other heavy computation work because it takes time to process a request and thereby blocks the single thread.

Node.js Console - REPL

Node.js comes with virtual environment called REPL (aka Node shell). REPL stands for `Read-Eval-Print-Loop`. It is a quick and easy way to test simple Node.js/JavaScript code.

To Start REPL, just type `node`

```
C:\Windows\system32\cmd.exe - node

C:\>node
> _
```

```
koushikkothagal — node — 80×24
koushikkothagal@Koushiks-MacBook-Pro-2 ~ % node -v
v13.2.0
koushikkothagal@Koushiks-MacBook-Pro-2 ~ % node
Welcome to Node.js v13.2.0.
Type ".help" for more information.
> 1 + 1
2
> var a;
undefined
> a = 10;
10
> a
10
> a * 2
20
> var b = a * 3;
undefined
> b
30
>
```

To execute JavaScript file from cmdline, use `node <.js-filename>`

```
//index.js
console.log("Hello, Node.js");
```

```
C:\NodeApp>node index.js
Hello, Node.js
```

Node.js Basics

Variables

to create a variable in Node.js, you need to make use of a reserved keyword `var`. You do not have to assign a data type, as **the compiler will automatically pick it**.

Datatypes

Node.js has various datatypes, categorized into Primitive and Non-Primitive datatypes.

Primitive Data Types are:

1. String
2. Number
3. Boolean
4. Null
5. Undefined

```
var str = "Satya";
var num = 10;
```

All the Datatypes are taken care by compiler – based on data we are assigning, we don't need to define datatype manually.

Non-Primitive Data Types are:

1. Object
2. Date
3. Array

```
var obj = {
    authorName: 'Ryan Dahl',
    language: 'Node.js'
}
```

Buffer

Node.js includes an additional data type called Buffer (not available in browser's JavaScript). Buffer is mainly used to store binary data, while reading from a file or receiving packets over the network.

Functions

A function can have attributes and properties also. It can be treated like a class in JavaScript.

```
function Display(x) {
    console.log(x);
}

Display(100);
```

Global Scope

In a browser, global scope is the **window** object.

```
window.location = "http://www.yoururl.com";
```

In Node.js, **global** object represents the **global scope**.

Node.js Modules

Module in Node.js is a simple or complex functionality organized in single or multiple JavaScript files which can be reused throughout the Node.js application.

Each module in Node.js has its own context, so it cannot interfere with other modules or pollute global scope. Also, each module can be placed in a separate .js file under a separate folder.

Node.js includes three types of modules:

1. **Core Modules**
2. **Local Modules**
3. **Third Party Modules**

1.Core Modules

Node.js is a lightweight framework. The core modules include bare minimum functionalities of Node.js. These core modules are compiled into its binary distribution and load automatically when Node.js process starts. However, you **need to import the core module** first in order to use it in your application.

The following table lists some of the important core modules in Node.js.

Core Module	Description
`http`	http module includes classes, methods and events to create Node.js http server.
`url`	url module includes methods for URL resolution and parsing.
`querystring`	querystring module includes methods to deal with query string.
`path`	path module includes methods to deal with file paths.
`fs`	fs module includes classes, methods, and events to work with file I/O.
`util`	util module includes utility functions useful for programmers.

In order to use Node.js core or NPM modules, you first need to import it using `require()` function as shown below.

```
var module = require('module_name');
```

As per above syntax, specify the module name in the `require()` function. The `require()` function will return an object, function, property or any other JavaScript type, depending on what the specified module returns.

The following example demonstrates how to use Node.js `http` module to create a web server.

```
var http = require('http');

var server = http.createServer(function(req, res){
  //write code here
});
server.listen(5000);
```

In the above example, require() function returns an object because http module returns its functionality as an object, you can then use its properties and methods using dot notation e.g. `http.createServer().`

2.Local Module

Local modules are modules created locally in your Node.js application. These modules include different functionalities of your application in separate files and folders. You can also package it and distribute it via NPM, so that Node.js community can use it.

For example, if you need to connect to MongoDB and fetch data then you can create a module for it, which can be reused in your application.

Writing Simple Module

Let's write simple logging module which logs the information, warning or error to the console.

log.js

```
var log = {
            info: function (info) {
                console.log('Info: ' + info);
            },
            warning:function (warning) {
                console.log('Warning: ' + warning);
            },
            error:function (error) {
                console.log('Error: ' + error);
            }
    };

module.exports = log
```

Loading Local Module - `app.js`

```
var myLogModule = require('./log.js');

myLogModule.info('Node.js started');
```

Run the above example using command prompt (in Windows) as shown below.

```
C:\> node app.js
Info: Node.js started
```

In the above example of logging module, we have created an object with three functions `- info(), warning() and error()`. At the end, we have assigned this object to `module.exports.` The module.exports in the above example exposes a log object as a module.

The `module.exports` is a special object which is included in every JS file in the Node.js application by default. Use **module.exports** or **exports** to expose a function, object or variable as a module in Node.js.

3.External Modules

You can use the external or 3rd party modules only by downloading them via **NPM**. These modules are generally developed by other developers and are free to use. Few of the best external modules are *express, react, gulp, mongoose, mocha etc*.

NPM - Node Package Manager

- Node Package Manager (NPM) is a command line tool that **installs, updates or uninstalls** Node.js packages in your application.
- It is also an **online repository** for open-source Node.js packages.
- The node community around the world creates useful modules and publishes them as packages in this repository.
- Official website: https://www.npmjs.com

NPM is included with Node.js installation. After you install Node.js, verify NPM installation by writing the following command in terminal or command prompt.

```
C:\NodeApp>npm -v
6.14.12
```

Install Package Locally

Use the following command to install any third party module in your local Node.js project folder.

```
C:\>npm install <package name>
```

For example, the following command will install ExpressJS into MyNodeProj folder.

```
C:\MyNodeProj> npm install express
```

All the modules installed using NPM are installed under **node_modules** folder. The above command will create ExpressJS folder under node_modules folder in the root folder of your project and install Express.js there.

Save package in package.json

Use `--save` at the end of the install command to add dependency entry into `package.json` of your application.

```
npm install express --save
```

Install Package Globally

NPM can also install packages globally so that all the node.js application on that computer can import and use the installed packages. NPM installs global packages into `/<User>/local/lib/node_modules` folder.

Apply `-g` in the install command to install package globally. For example, the following command will install ExpressJS globally.

```
C:\MyNodeProj> npm install -g express
```

```
## Update package
npm update express

## Uninstall package
npm uninstall express
```

Nodejs Examples – Module by Module

Before going to code, we must create a project to write a code. For that use `npm init &` Provide required input. It will create required files & package.json

```
C:\\Nodeapp> npm init
About to write to C:\NodeApp\package.json:
{
  "name": "nodeapp",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "node index.js"
  },
  "author": "Satya",
  "license": "ISC"
}
```

Node.js Server - HTTP module

Example 1: Create a Webserver

Node.js provides capabilities to create your own web server which will handle HTTP requests asynchronously.

```
// 1 - Import Node.js core module
var http = require('http');

// 2 - creating server
var server = http.createServer(function(req, res) {
  //Logic to handle requests
});

//3 - listen for incoming requests on port 5000
server.listen(5000);
console.log('Server Started ....')
```

```
c:\\NodeApp> node server.js
Server Started…
```

This is how you create a Node.js web server using simple steps. Now, let's see how to handle HTTP request and send response in Node.js web server.

Example 2 : Handle HTTP Request

The `http.createServer()` method includes request and response parameters which is supplied by Node.js.

- `request` object can be used to get information about the current HTTP request `e.g., url, request header, and data`.
- `response` object can be used to send a response for a current HTTP request.

```
// Import Node.js core module
var http = require('http');

//create web server
var server = http.createServer(function (req, res) {
    if (req.url == '/') { //check the URL of the current request

        // set response header
        res.writeHead(200, { 'Content-Type': 'text/html' });

        // set response content
        res.write('<html><body><p>This is home Page.</p></body></html>');
        res.end();

    }
    else if (req.url == "/student") {
        res.writeHead(200, { 'Content-Type': 'text/html' });
        res.write('<html><body><p>This is student Page.</p></body></html>');
        res.end();
    }
    else if (req.url == "/admin") {
        res.writeHead(200, { 'Content-Type': 'text/html' });
        res.write('<html><body><p>This is admin Page.</p></body></html>');
        res.end();
    }
    else
        res.end('Invalid Request!');

});

server.listen(5000); //6 - listen for any incoming requests

console.log('Node.js web server at port 5000 is running..')
```

In the above example,

- `req.url` is used to check the url of the current request and based on that it sends the response.
- To send a response, first it sets the response header using `writeHead()` method and then writes a string as a response body using `write()` method.
- Finally, Node.js web server sends the response using `end()` method.

Example 3 : Sending JSON Response

```
var http = require('http');

var server = http.createServer(function (req, res) {
    if (req.url == '/data') { //check the URL of the current request
            res.writeHead(200, { 'Content-Type': 'application/json' });
            res.write(JSON.stringify({ message: "Hello World"}));
            res.end();
    }
});

server.listen(5000);
```


Satya Kaveti

Node.js File System – 'fs' module

Node.js includes **fs** module to access physical file system. The fs module is responsible for all the asynchronous or synchronous file I/O operations.

Read files

```
fs.readFile(fileName [,options], callback)
```

```
var fs = require('fs');

fs.readFile('a.txt',
    function (err, data) {
                    if (err){
                        throw err;
                    }
    console.log(data);
});
```

```
fs.readFileSync('dummyfile.txt', 'utf8');
```

```
var fs = require('fs');

var data = fs.readFileSync('a.txt', 'utf8');
console.log(data);
```

Write to files

```
fs.writeFile(filename, data[, options], callback)
```

```
var fs = require('fs');

fs.writeFile('test.txt', 'Hello World!', function (err) {
                        if (err)
        console.log(err);
                        else
        console.log('Write operation complete.');
});
```

```
fs.appendFile('test.txt', 'Hello World!', callback)
```

```
var fs = require('fs');

fs.appendFile('test.txt', 'Hello World!', function (err) {
                        if (err)
        console.log(err);
                        else
        console.log('Append operation complete.');
});
```

Delete files : `fs.unlink(path, callback);`

Rename files : `fs.rename(oldPath, newPath, callback)`

Node.js Database – MySQL Module

You have to install MySQL driver to access a MySQL database with Node.js.

To download and install the "mysql" module, open the Command Terminal and execute the following:

```
npm install mysql
```

Exampale

```
var mysql = require('mysql');
var con = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "root",
  database:"webapp"
});

//Create Connection
/*
con.connect(function (err) {
  if (err) throw err;
  console.log("Connected!");
});

*/

con.connect(function (err) {
  if (err) throw err;
  con.query("SELECT * FROM user", function (err, result) {
    if (err) throw err;
    console.log(result);
  });
});
con.end;
```

```
C:\NodeApp>node index.js
[
  RowDataPacket {
    id: 1,
    username: 'satya',
    password: 'sasas',
    email: '123',
    status: 'active'
  },
  RowDataPacket {
    id: 2,
    username: 'satya',
    password: 'smlcodes',
    email: '111',
    status: 'active'
  },
  RowDataPacket {
    id: 3,
    username: 'satya',
    password: 'satya',
    email: 'smlcodes@gmail.com',
    status: 'active'
  }
]
```

Similarly, for MongoDB, we need to use 'mongodb' module.

```
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/MongoDatabase";
MongoClient.connect(url, function (err, db) {
  if (err) throw err;
  console.log("Database created!");
  db.close();
});

var http = require('http');
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/MongoDatabase";
MongoClient.connect(url, function (err, db) {
  if (err) throw err;
  db.collection("employees").findOne({}, function (err, result) {
    if (err) throw err;
    console.log(result.name);
    db.close();
  });
});
```

Events Module

https://www.tutorialsteacher.com/nodejs/nodejs-eventemitter

Node.js – Frameworks

There are various third party open-source frameworks available in Node Package Manager which makes Node.js application development faster and easy.

The following table lists frameworks for Node.js., Many more there… above are few of them.

Open-Source Framework	Description
Express.js	Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. This is the most popular framework as of now for Node.js.
Mojito	This HTML5 framework for the browser and server from Yahoo offers direct MVC access to the server database through the local routines. One clever feature allows the code to migrate. If the client can't run JavaScript for some reason, Mojito will run it on the server -- a convenient way to handle very thin clients.
Restify	Restify is a node.js module built specifically to enable you to build correct REST web services.
Chocolate.js	Chocolate is a simple webapp framework built on Node.js using Coffeescript.

Node.js – Differences

Difference between Node.js and AngularJS

Node.js	AngularJS
You have to install Node.js on your computer system to use it further for creating web or chat applications.	You have to add the AngularJS file just like any other JavaScript file to use it in applications. It doesn't need to be installed separately.
Node.js supports non-blocking input output I/O and follows an event driven architecture. It is used to create real-time applications such as instant messaging or chat apps.	AngularJS is completely written in JavaScript. It is mainly used to create single-page client-side applications.
Node.js is a platform built on the top of Google's V8 JavaScript engine.	AngularJS is an open source framework, follows the syntax of JavaScript and developed by Google.
Node.js is written in C, C++ and JavaScript languages.	AngularJS is written completely in JavaScript, but it is different from other web application frameworks like jQuery.
Node.js has a lot number of frameworks such as Express.js, Sails.js, Partial.js etc.	AngularJS itself is a web application framework of JavaScript.

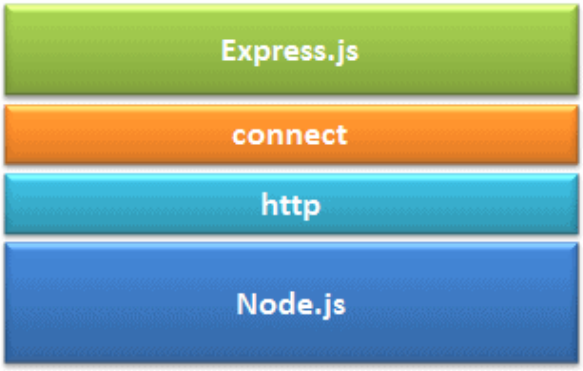
Ref.

https://www.tutorialsteacher.com/nodejs/nodejs-tutorials

Express.js

Express.js is a web application framework for Node.js. It provides various features that make web application development fast and easy which otherwise takes more time using only Node.js.

Express.js is based on the **Node.js** middleware module called `connect` which in turn uses `http` module. So, any middleware which is based on `connect` will also work with Express.js.



Advantages of Express.js

1. Makes Node.js **web application development fast and easy**.
2. Allows you to define routes of your application based on HTTP methods and URLs.
3. Includes various middleware modules which you can use to perform additional tasks on request and response.
4. Allows you to define an **error handling middleware**.
5. **Easy to serve static files** and resources of your application.
6. Allows you to create **REST API server**.
7. Easy to connect with databases such **as MongoDB, Redis, MySQL**

Installation

You can install express.js using **npm**. The following command will install latest version of express.js globally on your machine so that every Node.js application on your machine can use it.

```
npm install -g express
```

```
npm install express-generator -g (Optional – Recommended.)
```


Satya Kaveti

Express.js vs Node.js

Node.js	Express.js
It is used to build **server-side, input-output, event-driven** apps.	It is used to build **Web Applications** using approaches and principles of Node.js.
It is built on Google's V8 engine.	It is built on **the top of Node.js.**
Written in C, C++, JavaScript	Written in JavaScript
It is a Run-time platform or environment designed for server-side execution of JavaScript.	**It is a Framework,** based on Node.js.
Controllers are not provided. We need to write manually	Controllers are provided. We just directly use them.
Routing is not provided. need to write manually	Routing is provided.
It requires more coding time.	It requires less coding time.

Express.js Web Application

Express.js provides an easy way to create web server and render HTML pages for different HTTP requests by configuring routes for your application.

Express.js - Create Web Server

In Node.js we manually created server by writing code.

```
var http = require('http');

var server = http.createServer(function(req, res) {
  //Logic to handle requests
});

server.listen(5000);
```

But in Node.js we don't need to write that much code, to create server & request processing. We need to just import **express** module & use the predefined methods.

Example to Create new server

```
var express = require('express');
var app = express();

// define routes here..

var server = app.listen(5000, function () {
    console.log('Node server is running..');
});
```

1.we imported Express.js module using **require()** function.

```
var express = require('express');
```

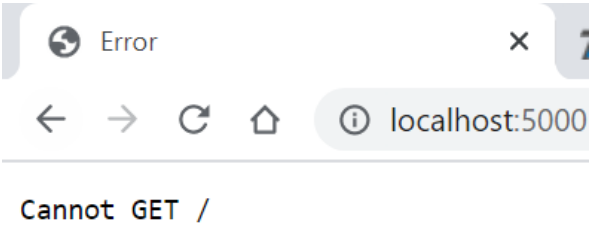
2. Then our application contains express.js related features. Now we need to create an express object, using **express();** constructor.

```
var app = express();
```

here the **app** object contains **methods** for routing HTTP requests, configuring middleware, rendering HTML views and registering a template engine.

3. The **app.listen()** function creates the Node.js web server at the specified **host and port**. It is identical to Node's **http.createServer.listen()** method.

4. Run the above example using **node app.js** command and point your browser to **http://localhost:5000**. It will display `Cannot GET /` because we have not configured any routes yet.



Express.js – Routing

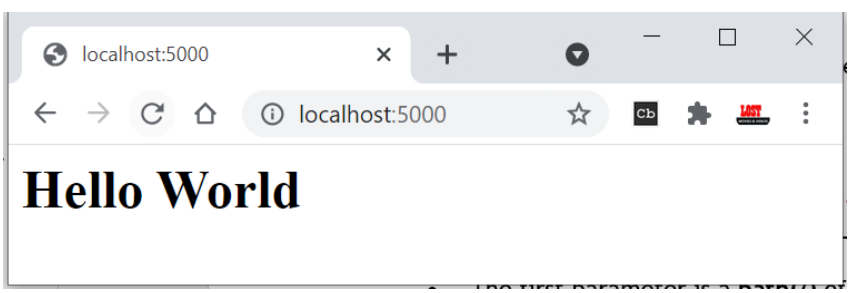
Use `app` object to define different routes of your application.

The **app** object includes *get(), post(), put() and delete()* methods to define routes for HTTP GET, POST, PUT and DELETE requests respectively.

```
var express = require('express');
var app = express();

app.get('/', function (req, res) {
  res.send('<html><body><h1>Hello World</h1></body></html>');
});

app.post('/post', function (req, res) {
  res.send('POST Request');
});

app.put('/put', function (req, res) {
  res.send('PUT Request');
});

app.delete('/delete', function (req, res) {
  res.send('DELETE Request');
});

var server = app.listen(5000, function () {
  console.log('Node server is running..');
});
```

- The first parameter is a **path(/)** of a route which will start after base URL.
- The callback function includes request and response object which will be executed on each request.

Run the above example using **node server.js** command and point your browser to *http://localhost:5000* and you will see the following result.



But for *get(), post(), put() and delete()* You still get **Cannot GET /put** because you are **not handling those requests properly.**

You need to send data to those request types. Next section will see how to send data using Forms.

Handle POST Request

you will learn how to handle HTTP POST request and get data from the submitted form.

To handle HTTP POST request in Express.js version 4 and above, you need to install middleware module called **body-parser**. This body-parser module parses the `JSON, buffer, string and url encoded data` submitted using HTTP POST request.

Install body-parser using NPM as shown below.

```
npm install body-parser --save
```

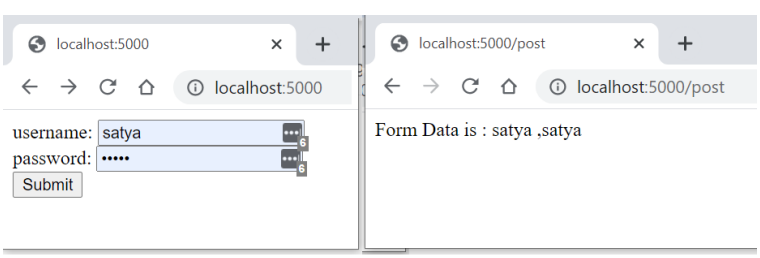
First, create `index.html` file in the root folder of your application and write the following code in it.

```
<form action="/post" method="post">
    username: <input name="username" type="text" /> <br />
    password: <input name="password" type="password" /> <br />
    <input type="submit" />
</form>
```

Create `app.js` file, import **body-parser** and get the POST request data as shown below.

```
var express = require('express');
var app = express();

//Import body parser
var bodyParser = require("body-parser");
app.use(bodyParser.urlencoded({ extended: false }));


//set Homepage - index.html
app.get('/', function (req, res) {
    res.sendFile(__dirname + '/index.html');
});


//process form data
app.post('/post', function (req, res) {
    var name = req.body.username + ' ,' + req.body.password;
    res.send('Form Data is : '+name);
});

var server = app.listen(5000, function () {
    console.log('Node server is running..');
});
```

In the above example, POST data can be accessed using **req.body**. The **req.body** is an object that includes properties for each submitted form.



Express.js - Serving Static Content

In this section, you will learn how to serve static resources like `images, css, JavaScript or other static files.`

Using **`express.static()`** method, you can server static resources directly by specifying the folder name where you have stored your static resources.

The following example serves static resources from the **public** folder under the root folder of your application

```
└──────public
        index.html
        signin.html
        logo.png
```

```
var express = require('express');
var app = express();

//setting middleware
app.use(express.static(__dirname + 'public')); //Serves resources from public folder


var server = app.listen(5000);
```

Express.js – Database Access

Node.js (Express.js on top of Node.js) supports all kinds of databases no matter if it is a relational database or NoSQL database.

To access the database from Node.js, you first need to install drivers for the database you want to use.

The following table lists important **relational databases** and respective drivers.

Relational Databases	Driver	NPM Command
MS SQL Server	**mssql**	`npm install mssql`
Oracle	**oracledb**	`npm install oracledb`
MySQL	**MySQL**	`npm install mysql`
PostgreSQL	**pg**	`npm install pg`
SQLite	**node-sqlite3**	`npm install node-sqlite`

The following table lists important **NoSQL databases** and respective drives.

NoSQL Databases	Driver	NPM Command
MongoDB	mongodb	`npm install mongodb`
Cassandra	cassandra-driver	`npm install cassandra-driver`
LevelDB	leveldb	`npm install level levelup leveldown`
RavenDB	ravendb	`npm install ravendb`
Neo4j	neo4j	`npm install neo4j`
Redis	redis	`npm install redis`
CouchDB	nano	`npm install nano`

Express.js – MySQL Connection

Install MySQL driver

```
npm install mysql --save
```

Example

```
var express = require('express');
var mysql = require('mysql');
var app = express();

//Create MySQL DB Connection
var connection = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: 'root',
    database: 'webapp'
});


app.get('/', function (req, res) {
    connection.connect(function (error) {
        if (!!error) {
            console.log(error);
        } else {
            connection.query('SELECT * FROM user ORDER BY id desc', function (err, rows) {
                if (err) {
                    console.log(err);
                } else {
                    res.send(rows);
                }
            });
        }
    });
});

var server = app.listen(5000, function () {
    console.log('Server is running..');
});
```

Express.js – MongoDB Connection

In order to access MongoDB database, we need to install MongoDB drivers

```
npm install mongodb --save
```

Connecting MongoDB

```
var MongoClient = require('mongodb').MongoClient;

// Connect to the db
MongoClient.connect("mongodb://localhost:27017/MyDb", function (err, db) {

     if(err) throw err;

     //Write databse Insert/Update/Query code here..
});
```

Insert Documents

```
var MongoClient = require('mongodb').MongoClient;

// Connect to the db
MongoClient.connect("mongodb://localhost:27017/MyDb", function (err, db) {

    db.collection('Persons', function (err, collection) {
        collection.insert({ id: 1, firstName: 'Steve', lastName: 'Jobs' });
        collection.insert({ id: 2, firstName: 'Bill', lastName: 'Gates' });
        collection.insert({ id: 3, firstName: 'James', lastName: 'Bond' });


        db.collection('Persons').count(function (err, count) {
            if (err) throw err;
            console.log('Total Rows: ' + count);
        });
    });

});
```

Select Query

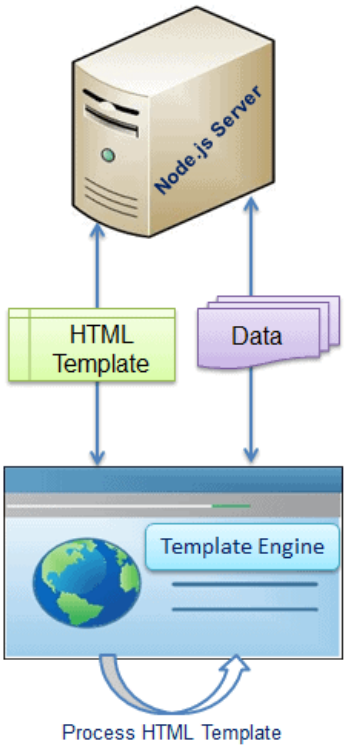
```
var MongoClient = require('mongodb').MongoClient;

// Connect to the db
MongoClient.connect("mongodb://localhost:27017/MyDb", function (err, db) {

    db.collection('Persons', function (err, collection) {

         collection.find().toArray(function(err, items) {
            if(err) throw err;
            console.log(items);
        });
    });
});
```

Similarly we do Update/Delete as well.

Node.js - Template Engines

Template engine helps us to create an HTML template with minimal code. Also, it can inject data into HTML template at client side and produce the final HTML.

The following figure illustrates how template engine works in Node.js.



Template engine produces the final HTML using **template** and **data** in client's browser.

The following is a list of important (but not limited) template engines for Node.js

- Jade
- Vash
- EJS
- Mustache
- Dust.js
- Nunjucks
- Handlebars
- atpl
- haml

React JS

Front-end development refers to what the end user (also commonly referred to as the "**client**") can see. In the most basic forms, front-end development consists of HTML, CSS, and JavaScript.

Eventually, developers decided that there must be a better way to manage all of that code, so they created libraries that could make life easier. **React** was one of those libraries.

React was created by Facebook and released to the public in May of 2013 and has been consistently maintained since then.

- React is used to build user interfaces (UI) on the front end.
- React is the **view** layer of an MVC application (Model View Controller)

React - Introduction

React is one of the most popular JavaScript libraries for **front-end web applications**.

Interactive websites need to update the DOM (Document Object Model) each time a change happens. This process is generally resourceful and slow.

Compared to other libraries that manipulate the DOM, React uses a **Virtual DOM**, allowing to update only the parts of the website that have changed. This increases the speed of updates dramatically, as modern web applications may contain thousands of elements.

Installation

To work with React, we don't need to install anything. We just need to add following `js` files.

- React - the React top level API
- React DOM - adds DOM-specific methods
- Babel - a JavaScript compiler that lets us use ES6+ in old browsers

First, we need to add the React library as two **script** tags to the **head** of our HTML document:

```
<script src="https://unpkg.com/react@16/umd/react.development.js" crossorigin></script>
<script src="https://unpkg.com/react-dom@16/umd/react-dom.development.js" crossorigin />
```

To enable the use of **JSX, use babel**. **JSX** is a syntax extension to JavaScript.

```
<script src="https://unpkg.com/babel-standalone@6/babel.min.js"></script>
```

Hello, World – Using index.tml

After adding the required script tags, we can start building our React app!

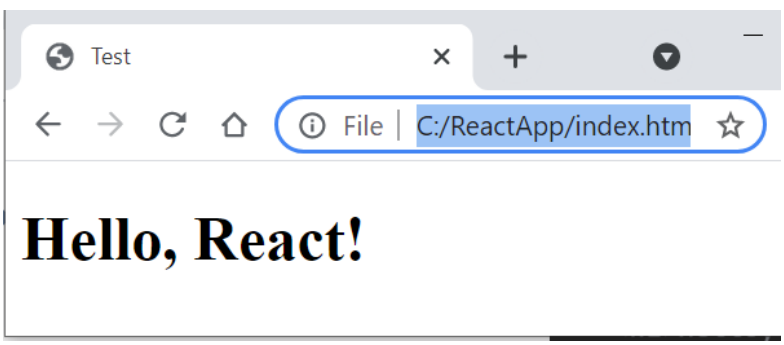
We need to add a container to display something using React.

```
<div id="container"></div>
```

ReactDOM.render(data, element)

```
<!DOCTYPE html>
<html>
    <head>
        <title>Test</title>
        <script src="https://unpkg.com/react@16/umd/react.development.js" crossorigin/>
        <script src="https://unpkg.com/react-dom@16/umd/react-
dom.development.js" crossorigin></script>
        <script src="https://unpkg.com/babel-standalone@6/babel.min.js"></script>
    </head>
    <body>
        <div id="container"></div>
        <script type="text/babel">

        ReactDOM.render(
          <h1>Hello, React!</h1>,
          document.getElementById('container')
        )

        </script>
    </body>
</html>
```

The code finds the **container** and adds the **h1** heading to it.



HelloWorld – Using Class

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/react@16/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@16/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
  </head>
```

```
  <body>
    <div id="root"></div>

    <script type="text/babel">
      class App extends React.Component {
        render() {
          return <h1>Hello Satya!</h1>
        }
      }

      ReactDOM.render(<App />, document.getElementById('root'))
    </script>
  </body>
</html>
```

In above we are **`return`** **single line** content, if we have **multiple lines** - we need to use one of below.

1.using ( ) brackets

```
    render() {
        return (
        <div className="App">
            <h1>Hello, React!</h1>
         </div>
        );
      }
```

2.using [ ] - as Array of elements

```
      render() {
        return [
          <ChildA key="key1" />,
          <ChildB key="key2" />,
          <ChildC key="key3" />,
        ];
      }
```

React - Create React App

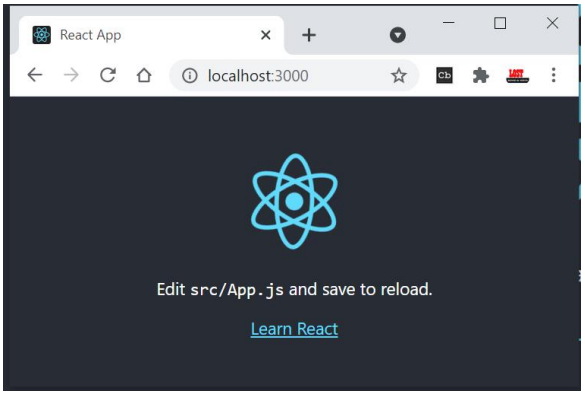
In the previous lesson we learned how to add React to a simple HTML document using the script tags. However, real web apps have a different scale, contain multiple files, use 3rd party libraries, etc.

Facebook has created a handy tool called **`create-react-app`** that makes it easy to setup a React project with just a simple command. **Create React App** allows us to focus on the code, rather than installing and configuring different tools.

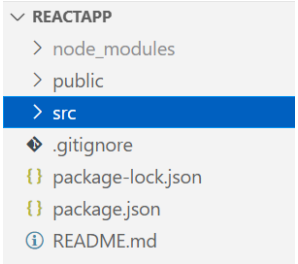
To get started, make sure you have a recent version of Node installed on your machine.
Run the following commands in the Terminal to create and start a React app called "**ReactApp**":

```
npx create-react-app reactapp
cd reactapp
npm start
```

This will install all the required dependencies, configure and start the project on **localhost:3000**. This is the default output of our project in the browser:



Folder Structure



`/public` folder contains files related to how the application will display on the client, the most important of those being **index.html**, which is the HTML template of our page.

`/src` folder contains all of the `JavaScript, CSS, and image files` that will be compiled into a bundle file and injected into **index.html.**

The two files below are the only critical files:

- `index.js:` This file is the entry point into our application. In our code, a method called `ReactDOM.render()` is used to find an element with `id="root"` in the HTML and add content to it.

- `app.js`: This file is the main **component** that will be rendered to the DOM, which currently includes the React logo image and the default text, that we see in the output.

Webpack creates a "bundle" file containing the content of multiple files that need to be "bundled" together and it is all added together into a single file.

Customizing React App

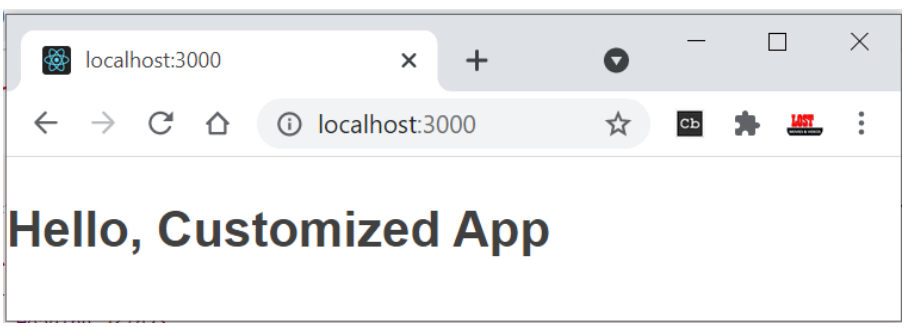
Go ahead and delete all the files out of the `/src` directory, and we'll create our own boilerplate file without any bloat. We'll just keep **`index.css`** and **`index.js`**.

Now in **`index.js`**, we're importing `React, ReactDOM,` and the `CSS file`.

```
import React from 'react'
import ReactDOM from 'react-dom'
import './index.css'

class App extends React.Component {
    render() {
        return (
        <div className="App">
            <h1>Hello, Customized App</h1>
         </div>
        );
      }
}

ReactDOM.render(<App />, document.getElementById('root'));
```

In **`index.html`**, just place only & **`start`** the Application

```
<div id="root"></div>
```



If we use `import React, {Component} from 'react',` No need to write complete extends class name

```
import React, {Component} from 'react'

class App extends Component {

}
```

React - JSX - JavaScript + XML

```
ReactDOM.render(
  <h1>Hello, React!</h1>,
  document.getElementById('root')
);
```

We will start with the `<h1>Hello, React!</h1>` element.

As you can see, the element is not in quotes to represent a string. It's like an HTML element, however we use it right in the JavaScript code!
This is called **JSX**, and it is a syntax extension to JavaScript. It allows us to build UI elements right in the JavaScript code!

The code calls React's **render** method, and passes it two arguments, a **JSX element** and **a container**. The `render` method displays the provided element in the container, which, in our case, is the HTML element with `id="root".`

Expressions in JSX

We can use any JavaScript expression inside JSX using curly braces.

For example:

```
const name = "David";
const el = <p>Hello, {name}</p>;

ReactDOM.render(
el,
document.getElementById('root')
);
```

Attributes in JSX

We can specify attributes using quotes, just like in HTML:

```
<div id="name"></div>
```

When using a JavaScript expression as the attributes value, the quotes should not be used:

```
<div id={user.id}></div>
```

Finally, When the JSX expressions are compiled, they are converted into JavaScript **objects**, representing React elements.

React then uses these elements to build the corresponding HTML DOM and display it in the browser.

Virtual DOM

We learned in the previous part that React updates only the elements that are necessary.
This allows React apps to be much faster than apps built with other front-end technologies.

But how does React achieve that?
React uses a **Virtual DOM**, which is a lightweight representation of the DOM.
When an element gets changed, it is first updated in the Virtual DOM. That process is fast, as the virtual DOM is represented by simple objects.

After that, React compares the Virtual DOM to its previous state and only applies the DOM updates necessary to bring the DOM to the desired state.

React – Components

So far, we've created one component - the `App` component. Most React apps have many small components, and everything loads into the main `App` component.

In React, there are two types of components that you can use:

- **Functional Components**
- **Class Components.**

Functional Components

A functional component is a simple JavaScript function:

```
function Hello() {
  return <h1>Hello world.</h1>;
}
```

The code above defined a functional component called **Hello**, that returns a simple React element.

Notice that the name of the functional component begins with a **capital letter**. This is absolutely critical. If we start the name of a component with a lowercase letter, the browser will treat our component like a regular HTML element instead of a Component.

In order to display the component, we need to create the corresponding JSX element, by calling the function.

For example, for our user-defined component **Hello**:

```
const el = <Hello/>;
```

Now, we can use our user-defined element and render it on the page:

```
function Hello() {
  return <h1>Hello world.</h1>;
}

const el = <Hello />;
ReactDOM.render(
  el,
  document.getElementById('root')
);
```

Remember, all component names need to start with a capital letter.

Class Components

Class components are typically used when there are more advanced user interactions, like forms, and animations.

All class components need to extend the **React.Component** class. Class components need to have a **render()** method, which is in charge of telling what the page should show.

Remove the **App** class from **index.js**, place it in new file called **app.js**

```
//app.js
import React, {Component} from 'react'

class App extends Component {
    render() {
        return (
        <div className="App">
            <h1>Hello, Customized App</h1>
         </div>
        );
      }
}
export default App
```

Import in **index.js**

```
import React from 'react'
import ReactDOM from 'react-dom'
import './index.css'
import App from './app'

 //Code Removed

ReactDOM.render(<App />, document.getElementById('root'));
```

We export the component as **App** and load it in **index.js.**

In **index.html**, just place only & **start** the Application

```
<div id="root"></div>
```

Example Simple Component

simple component, which is a function. This component doesn't use the `class` keyword. Let's take our **Table** and make two simple components for it - a **TableHeader**, and a **TableBody**.

We're going to use ES6 arrow functions to create these simple components.

First, the `TableHeader`

```
const TableHeader = () => {
    return (
      <thead>
        <tr>
          <th>Name</th>
          <th>Job</th>
        </tr>
      </thead>
    )
  }
```

Then the `TableBody` .

```
  const TableBody = () => {
    return (
      <tbody>
        <tr>
          <td>Charlie</td>
          <td>Janitor</td>
        </tr>
        <tr>
          <td>Mac</td>
          <td>Bouncer</td>
        </tr>
      </tbody>
    )
  }
```

Now our Table file will look like this. Note that the `TableHeader` and `TableBody` components are all in the same file and being used by the Table class component.

```
const TableHeader = () => { ... }
const TableBody = () => { ... }

class Table extends Component {
  render() {
    return (
      <table>
        <TableHeader />
        <TableBody />
      </table>
    )
  }
}
```

Example Class Component

We are created Table component to display table data in **table.js** - and export as **Table** component

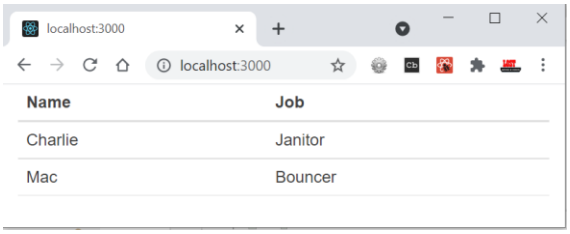
```
import React, {Component} from 'react'

class Table extends Component {
  render() {
    return (
      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Job</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>Charlie</td>
            <td>Janitor</td>
          </tr>
          <tr>
            <td>Mac</td>
            <td>Bouncer</td>
          </tr>
        </tbody>
      </table>
    )
  }
}
export default Table
```

Import Table in **app.js** - pass it to index.html

```
import React, {Component} from 'react'
import Table from './table'

class App extends Component {
  render() {
    return (
      <div className="container">
        <Table />
      </div>
    )
  }
}
export default App
```

In **index.html**, just place only & **start** the Application

```
<div id="root"></div>
```



React – Data Handling

To handle data in React, we do with properties, referred to as **props**, and with **state**

Props

Functional components can accept arguments, similar to JavaScript functions. These arguments are called **props**, and represent an object.

For example, we can use props in our Hello component:

```
function Hello(props) {
  return <p>Hello, {props.name}!</p>;
}
```

Props can be accessed in class components using **`this.props`**.

```
class Hello extends React.Component {
  render() {
    return <p>Hello, {this.props.name}!</p>;
  }
}
```

An important thing to consider is that props are **read-only**, meaning components cannot modify their props.

State

Many web apps need their components to change their data, for example, after user interaction (clicking a button, submitting a form, etc.).However, **props** cannot be changed.

In order to allow components to manage and change their data, React provides a feature called **state**. **State** is an object that is added as a property in class components.

State is just a simple object, that contains **`key:value`** pairs.Similar to props, the values can be accessed using **this.state**.

```
class Hello extends React.Component {
  state = {
    name: "James"
  }
  render() {
    return <h1>Hello {this.state.name}.</h1>;
  }
}
```

Changing State

```
this.setState({
name: "James",
age: 25
});
```

Lifecycle Methods

Enough, from now onwards read

https://www.w3schools.com/react/react_props.asp

Ref.

https://www.sololearn.com/learning/1097/3370/7610/2

Best

https://www.taniarascia.com/getting-started-with-react/